

Study Notes for Operating Systems (ACAPC1522N)

Course Overview

- **Course Type:** Minor
- **Semester:** 5th
- **Credit Weightage:** Theory - 4, Practicals - 2
- **Batch:** 2022 onwards

Course Objective:

1. **Introduction to Operating System Concepts:** Understanding processes, threads, scheduling, synchronization, deadlocks, memory management, file and I/O subsystems, and protection mechanisms.
2. **Design and Development:** Focus on the issues in OS design and development.
3. **UNIX Familiarization:** Basic UNIX commands, system calls for process management, inter-process communication, and I/O operations.

Course Outcomes:

- **Access Control:** Ability to manage and control access to systems and files.
- **System Components:** Knowledge of computer components and their roles.
- **Troubleshooting:** Skills to diagnose and fix user problems in standard OS environments.
- **Practical Knowledge:** Insight into how programming languages interact with OS.

Module 1: Introduction to Operating Systems

1.1 What is an Operating System?

- Definition: Software that manages hardware resources and provides common services for computer programs.
- **Examples:** Windows, macOS, Linux, Android.

1.2 Types of Operating Systems

- **Batch Processing**
- **Multiprogramming**
- **Multitasking**
- **Real Time**
- **Distributed**
- **Network OS**

1.3 Functions of OS

- Process Management
- Memory Management
- File Management

- I/O System Management
- Secondary Storage Management
- Protection and Security

Module 2: Process Management

2.1 Processes and Threads

- **Process:** A program in execution.
- Example: Running a text editor.
- **Thread:** Smallest unit of CPU utilization within a process.
- Example: Multiple threads in a web browser for different tabs.

2.2 Scheduling

- **Algorithms:** First-Come-First-Served (FCFS), Shortest Job Next (SJN), Priority Scheduling, Round Robin (RR).
- **Purpose:** Efficient utilization of CPU, fairness in process execution.

2.3 Synchronization

- **Critical Section Problem:** Ensuring mutual exclusion when processes access shared resources.
- **Tools:** Semaphores, Monitors, Mutex locks.

2.4 Deadlocks

- **Definition:** A situation where a set of processes are blocked because each process holds a resource and is waiting to acquire a resource held by another process.
- **Strategies:** Deadlock prevention, avoidance, detection, and recovery.

Module 3: Memory Management

3.1 Memory Hierarchy

- Cache, Main Memory, Secondary Storage.

3.2 Memory Management Techniques

- **Paging:** Dividing memory into fixed-size blocks (pages).
- **Segmentation:** Memory management where address space is divided into variable size segments.
- **Virtual Memory:** Allows a program to use more memory than physically available by using disk space.

3.3 Swapping

- Moving processes between main memory and secondary storage to free up memory.

Module 4: File and I/O Subsystems

4.1 File Systems

- **Structure:** File directories, file attributes, file operations.
- **Types:** FAT, NTFS, ext4, etc.

4.2 I/O Management

- **Buffering:** Temporary storage of data to match the speed of different devices.
- **Caching:** Keeping frequently accessed data in memory.

Module 5: Protection and Security

5.1 Protection

- **Mechanisms:** Access control lists (ACLs), capabilities, protection domains.

5.2 Security

- **Authentication:** Verifying identity (passwords, biometrics).
- **Authorization:** Permissions and access rights.
- **Encryption:** Protecting data in transit and at rest.

Module 6: Unix and System Calls

6.1 Basic Unix Commands

- `ls`, `cd`, `mkdir`, `rm`, `cp`, `mv`, etc.

6.2 System Calls

- **Process Management:** `fork()`, `exec()`, `wait()`.
- **Inter-Process Communication:** Pipes, shared memory, message queues.
- **I/O Operations:** `read()`, `write()`, `open()`, `close()`.

6.3 Shell Scripting Basics

- Writing simple scripts to automate tasks.

These notes provide a structured overview of the Operating Systems course, covering key concepts with examples to enhance understanding. Remember, practical sessions will involve applying these concepts in real UNIX environments, so familiarity with command line interfaces and basic programming will be beneficial.

UNIT – I: Introduction to Operating Systems

1. Types of Operating Systems:

- **Simple Batch Systems:** Early systems where jobs were submitted in batches to be processed sequentially without user interaction.
Example: Punch card systems in the 1960s.
- **Multi-programmed Systems:** Multiple jobs are kept in memory at the same time, allowing CPU to switch between jobs when one is waiting for I/O.
Example: Mainframes where CPU utilization is maximized.
- **Time-shared Systems:** Users interact with the system through terminals, with the CPU dividing time among multiple users or processes.
Example: UNIX, where each user gets a time slice for processing.
- **Personal Computer Systems:** OS designed for single user interaction, focusing on user interface and ease of use.
Example: Windows, macOS.
- **Parallel Systems:** Systems with multiple CPUs or cores working together to complete tasks more efficiently.
Example: Modern servers and supercomputers.
- **Distributed Systems:** Collection of independent computers that appear to its users as a single coherent system.
Example: Cloud computing platforms like AWS.
- **Real-Time Systems:** OS that must guarantee response within strict time constraints.
Example: Air traffic control systems, medical equipment.

2. System Components:

- **CPU:** Executes instructions.
- **Memory:** Stores instructions and data.
- **I/O Devices:** For input and output operations.
- **File System:** Manages files on secondary storage.

3. Operating System Services:

- **Program Execution:** OS loads, executes, and manages programs.
- **I/O Operations:** Control devices for input and output.
- **File-System Manipulation:** Manage files and directories.
- **Communications:** Allows processes to exchange messages or share memory.
- **Error Detection:** Detects and recovers from errors.

****4. System Calls:****

- ****System Calls:**** Interface between the OS and the user program for services like process control, file management, etc.
- ****fork():**** Creates a new process by duplicating the calling process.
- ****exit():**** Terminates the calling process.
- ****wait():**** Makes a process wait until one of its children terminates.
- ****waitpid():**** Similar to wait but waits for a specific child process.
- ****exec():**** Replaces the current process image with a new process image.

****5. Process and CPU Scheduling:****

- ****Process Concepts:**** A process is a program in execution, including the program code and its current activity.
- ****Scheduling:**** Managing the order in which processes access the CPU.
- ****Scheduling Criteria:**** CPU utilization, throughput, turnaround time, waiting time, response time.
- ****Algorithms:****
 - First-Come-First-Served (FCFS)
 - Shortest Job Next (SJN)
 - Priority Scheduling
 - Round Robin (RR)
 - Multilevel Queue Scheduling
- ****Multiple-Processor Scheduling:**** Scheduling processes in environments with multiple CPUs or cores.

UNIT – II: Process Management and Synchronization

****1. Deadlocks:****

- ****System Model:**** Resources and processes interacting where multiple processes might request and hold resources leading to deadlocks.
- ****Characterization:**** Necessary conditions for deadlock (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait).
- ****Handling Methods:****
 - ****Prevention:**** Ensure at least one of the four conditions cannot hold.
 - ****Avoidance:**** Use algorithms like Banker's Algorithm to avoid entering unsafe states.
 - ****Detection:**** Use algorithms to detect deadlocks periodically.
 - ****Recovery:**** Strategies to recover from a deadlock, like process termination or resource preemption.

****2. Process Management and Synchronization:****

- ****Critical Section Problem:**** Ensuring that only one process can execute its critical section at a time.

- ****Solutions:**** Mutual exclusion, progress, and bounded waiting.

- ****Synchronization Hardware:****

- ****TestAndSet():**** Atomic operation for mutual exclusion.

- ****Swap():**** Another hardware solution for mutual exclusion.

- ****Semaphores:****

- ****Counting Semaphores:**** Generalization of binary semaphores.

- ****Binary Semaphores:**** Also known as mutex locks.

- ****Classical Problems of Synchronization:****

- ****Producer-Consumer:**** Synchronization between producers and consumers of a buffer.

- ****Readers-Writers:**** Multiple readers can access data simultaneously, but writers must have exclusive access.

- ****Dining Philosophers:**** Philosophers picking up chopsticks (resources) without causing a deadlock.

- ****Critical Regions:****

- A higher-level abstraction for handling critical sections.

- ****Monitors:****

- High-level synchronization construct that provides a convenient and effective mechanism for process synchronization.

These notes provide a comprehensive overview of key concepts in operating systems, focusing on process management, CPU scheduling, deadlock handling, and synchronization techniques. They include definitions, explanations, and examples to facilitate understanding.

UNIT III: Interprocess Communication Mechanisms and Memory Management

****Interprocess Communication Mechanisms (IPC)****

****1. IPC on a Single Computer System:****

- ****Shared Memory:**** Processes share a region of memory. This method is fast since no data copying is involved.

- ***Example:** Two processes can access a shared memory segment to update and read a counter value in real-time.

- ****Pipes:**** Unidirectional flow of data between related processes.

- ***Example:** A shell pipeline where the output of one command (`ls`) is piped to

another (`grep`).

- **FIFOs (Named Pipes):** Similar to pipes but can be used for communication between unrelated processes.
 - *Example:* A server writes data to a named pipe, and multiple clients can read from this pipe at different times.

2. IPC Between Processes on Different Systems:

- **Sockets:** Facilitate communication over networks.
 - *Example:* A client application on one computer can communicate with a server application on another using TCP/IP sockets.
- **Remote Procedure Calls (RPC):** A form of inter-process communication where a program causes a procedure to execute in another address space.
 - *Example:* NFS (Network File System) uses RPC to allow remote file access.

3. Detailed Mechanisms:

- **Message Queues:** Processes communicate by sending messages to and receiving from queues.
 - *Example:* In banking systems, different processes might use message queues to manage transaction requests.
- **Signals:** Software interrupts delivered to a process to notify an event has occurred.
 - *Example:* The SIGTERM signal can be sent to a process to request termination.

Memory Management and Virtual Memory

1. Logical vs. Physical Address Space:

- **Logical Address Space:** Generated by the CPU, managed by the MMU (Memory Management Unit).
- **Physical Address Space:** Actual memory locations in RAM.

2. Swapping:

- Moving entire processes in and out of main memory to disk to free up space for other processes.

3. Contiguous Memory Allocation:

- Memory is allocated in one contiguous block.
- *Drawbacks:* External fragmentation.

4. Paging:

- Memory is divided into fixed-size blocks called pages. Logical address space is divided into pages, and physical memory into frames.
- **Page Table:** Maps logical page numbers to physical frame numbers.
- *Example:* A process might have its first page mapped to frame 5, second to frame 10, etc.

5. Segmentation:

- Memory is divided into segments of different sizes, each representing a logical grouping (like code, data, stack).

- **Segment Table:** Maps segment numbers to physical addresses.

6. Segmentation with Paging:

- Combines segmentation and paging to address both logical and physical memory allocation issues.

7. Demand Paging:

- Pages are loaded into memory only when they are needed, reducing the initial memory footprint.

- **Page Fault:** When a page not in memory is accessed.

8. Page Replacement:

- When memory is full, a page must be selected to be replaced.

- **Algorithms:**

- **FIFO (First In First Out):** Replace the oldest page.

- **LRU (Least Recently Used):** Replace the page that has been used least recently.

- **Optimal Algorithm:** Replace the page that will not be used for the longest time in the future (used for theoretical purposes).

UNIT IV: File System Interface and Operations

File System Interface

1. Access Methods:

- **Sequential Access:** Read/write sequentially.

- **Direct Access:** Access any part of the file directly by specifying a block number.

2. Directory Structure:

- **Single-Level:** All files in one directory.

- **Two-Level:** User directories with system-wide files.

- **Tree-Structured:** Hierarchical organization of directories and files.

- **Acyclic-Graph:** Allows sharing of files or directories.

3. Protection:

- **Access Control:** Permissions (read, write, execute).

- **Encryption:** For data security.

4. File System Structure:

- **Superblock:** Contains metadata about the file system.

- **Inode:** Contains metadata about files like permissions, ownership, etc.

- **Data Blocks:** Where the actual file contents are stored.

5. Allocation Methods:

- **Contiguous Allocation:** Files occupy contiguous blocks.

- **Linked Allocation:** Files are linked lists of blocks.
- **Indexed Allocation:** An index block contains pointers to data blocks.

6. Free-space Management:

- **Bit Vector:** Each bit represents a block, 0 for free, 1 for allocated.
- **Linked List:** Free blocks linked together.
- **Grouping:** First free block contains pointers to several free blocks.

7. Usage of System Calls:

- **open():** To open a file for reading or writing.
- **create():** To create a new file.
- **read(), write():** For reading from or writing to a file.
- **close():** To close the file.
- **lseek():** To change the file offset.
- **stat():** To retrieve file status information.
- **ioctl():** For device-specific operations.

These notes provide an overview and examples for each section, making it easier to understand and memorize the key concepts involved in operating systems.